Arun Vemula

CPP final REPORT.pdf

SRM University AP Amravati

Document Details

Submission ID
trn:oid:::8044:156829592

Submission Date
Nov 2, 2026, 8:45 AM GMT+5:30

Download Date
Nov 2, 2026, 8:47 AM GMT+5:30

File Name
CPP final REPORT.pdf

File Size
605.8 KB

14 Pages

2,555 Words

13,138 Characters

0% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

1 Not Cited or Quoted 0%
Matches with neither in-text citation nor quotation marks

0 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **1** Not Cited or Quoted 0%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

0% Internet sources
0% Publications
0% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1 Internet

msuniversity.ac.in **<1%**



SRM UNIVERSITY AP

PROJECT REPORT

E-Voting System

1

COURSE NAME : Object Oriented Programming Using C++

COURSE CODE : *CSE 202*

SUBMITTED TO:

Dr. Kunal Biswas

SUBMITTED BY:

S.NO	NAME OF THE STUDENT	ROLL.NO
1	SANDEEP	AP25110010663
2	BHARAT	AP25110010617
3	ARUN	AP25110010429

Abstract

This is a console application using C++17 to simulate a basic election. It has two sections for the user and administrator. The user can register, log in, view candidates and cast one verified vote. The administrator can manage the voters and candidates, manage the election statuses and view statistics and results after voting has ended.

The project uses OOP to separate concerns into classes. The STL containers, searching, sorting, exceptions and text file storage are all used. The data is stored locally, so there is no need for an internet connection. The code will have checks to prevent duplicate voting, invalid candidates, invalid election statuses, corruption and other invalid inputs. This project is intended as an educational tool.

1. Introduction

An election involves multiple actions such as registering voters, managing candidates, verifying users, recording votes and generating results. Implementing these actions in an application helps to understand how programming concepts can be used together.

This project simulates a local election using C++17. It has a simple election status cycle with NOT STARTED, OPEN and CLOSED. Only voters can vote during the OPEN period and results are available after the election has CLOSED. The voting history is kept separate from the voters to simulate a secret ballot.

2. Problem Statement

The main aim of this project is to create a structured application to simulate a basic election with rules such as verifying users, only 1 vote per user, valid candidates and managing the election statuses.

The information must also be stored, and the application should be able to handle invalid or corrupt inputs.

3. Objectives

Create a full multi-file C++17 console application using OOP.

Separate out concerns with classes representing registration, verification, voting and counting.

Enforce rules such as one vote per user, valid candidates, verification and only being able to vote during the OPEN status.

Store data in text files and ensure data can be recovered safely if missing or corrupt.

Use defensive programming, validation and exceptions.

Use searching, sorting and STL containers on realistic data.

Have a simple and easy-to-use console interface.

4. Project Scope

4.1 Inclusions

Registration, Login, View Profile, View Candidates, Verified Vote, Generate Receipt, Logout.

Managing candidates and voters by the administrator.

Managing the election statuses with NOT STARTED, OPEN and CLOSED.

Viewing election statistics, turnout information, calculating results, handling winning/tied cases and displaying an ASCII bar chart.

Storing data using 6 text files, audit log, checking for data integrity, reset and restore demo data.

4.2 Exclusions

No internet connection, government databases or real elections.

No real verification of users such as ID cards or biometrics.

No real encryption or hardware security.

Only one election can be run at once and no advanced voting methods such as ranked choice or approval.

No GUI or multi-machine voting.

5. System Requirements

5.1 Software

A C++17-compatible compiler.

C++ Standard Library.

A console/terminal.

Access to file system to store text data.

5.2 Hardware

A computer that can run a C++17 compiler.

Enough storage to store source code and text data.

A keyboard and screen.

6. System Design

This application uses a layered approach to help manage the code. The model layer contains classes representing users, voters, administrators, candidates, votes and election status. The logic layer (mainly the VotingSystem class) contains the rules for the election. The data is stored using the FileManager and the user interface is handled by the ConsoleUI.

This helps make the code more readable and manageable. The voting rules aren't reliant on the console menu. This means that in the future the core logic could be connected to another interface.

7. Modules

7.1 User & Verification

User is an abstract base class for the users. Voter and Admin classes inherit from this. Both classes contain user information such as the user ID, name and password information. Each class has its own role and profile.

7.2 Candidate Management

The administrator can add, edit, delete, view and search for candidates. This is kept as separate data so the voting can verify the candidate.

7.3 Voter Management

The administrator can add, delete, list, sort and search for voters. The voters are stored in a map using their ID as the key.

7.4 Election Management

The election has 3 statuses: NOT STARTED, OPEN and CLOSED. The Election class manages starting, stopping and resetting the election to ensure invalid statuses aren’t chosen.

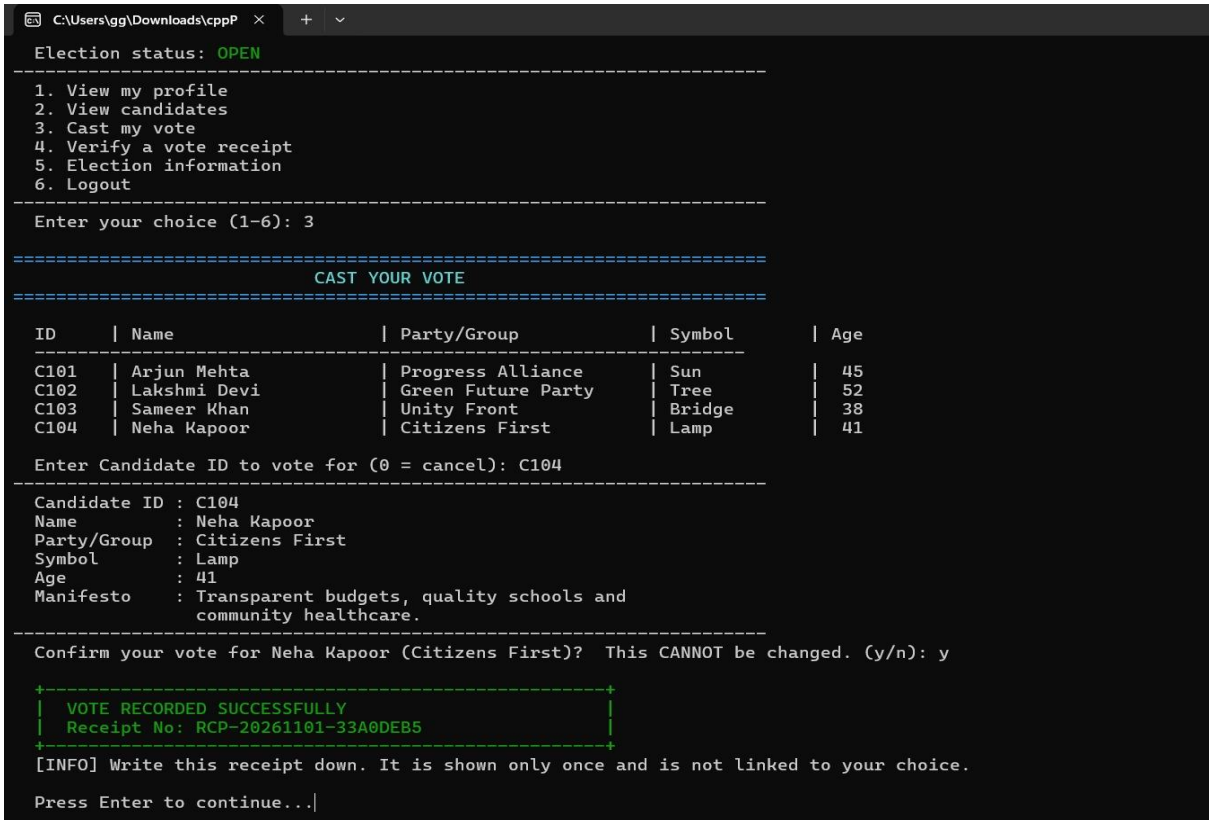
7.5 Voting

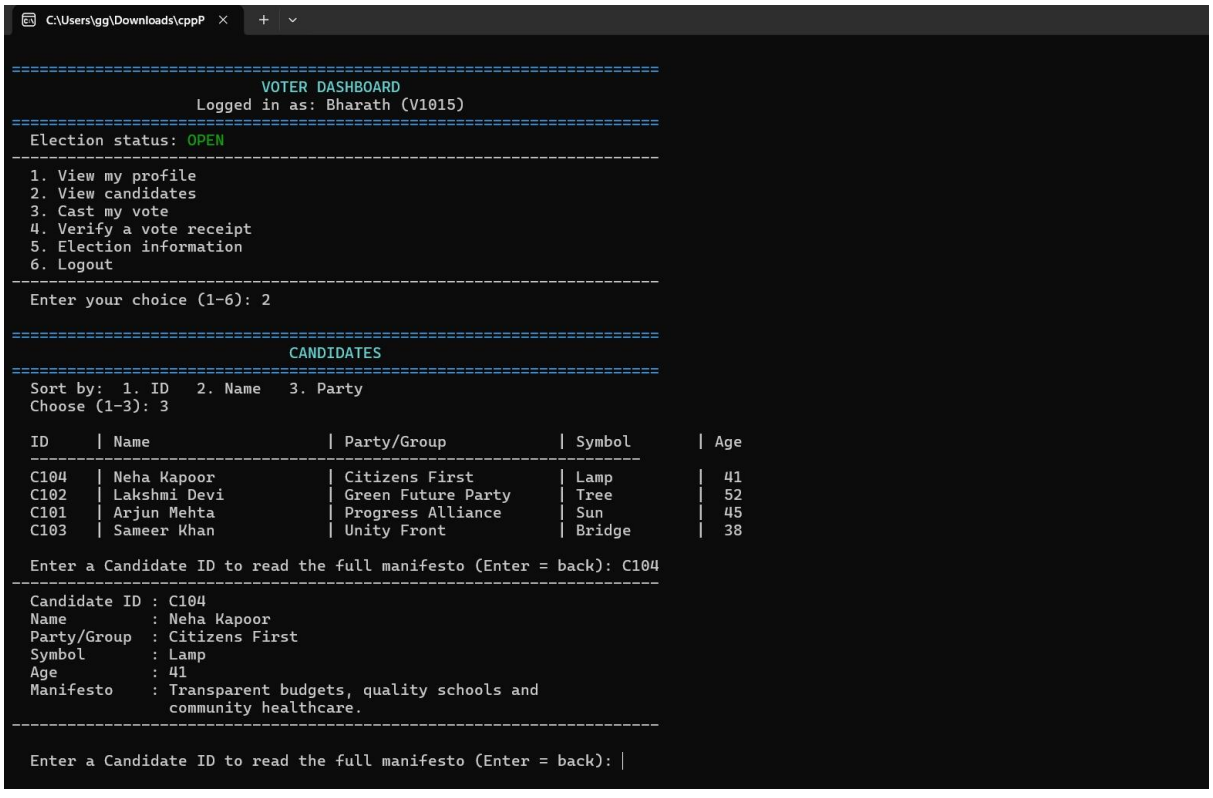
The user must be verified and meet the voting criteria. The candidate is checked and the user is prompted to confirm their choice. Once the ballot has been added the user is marked as having voted.

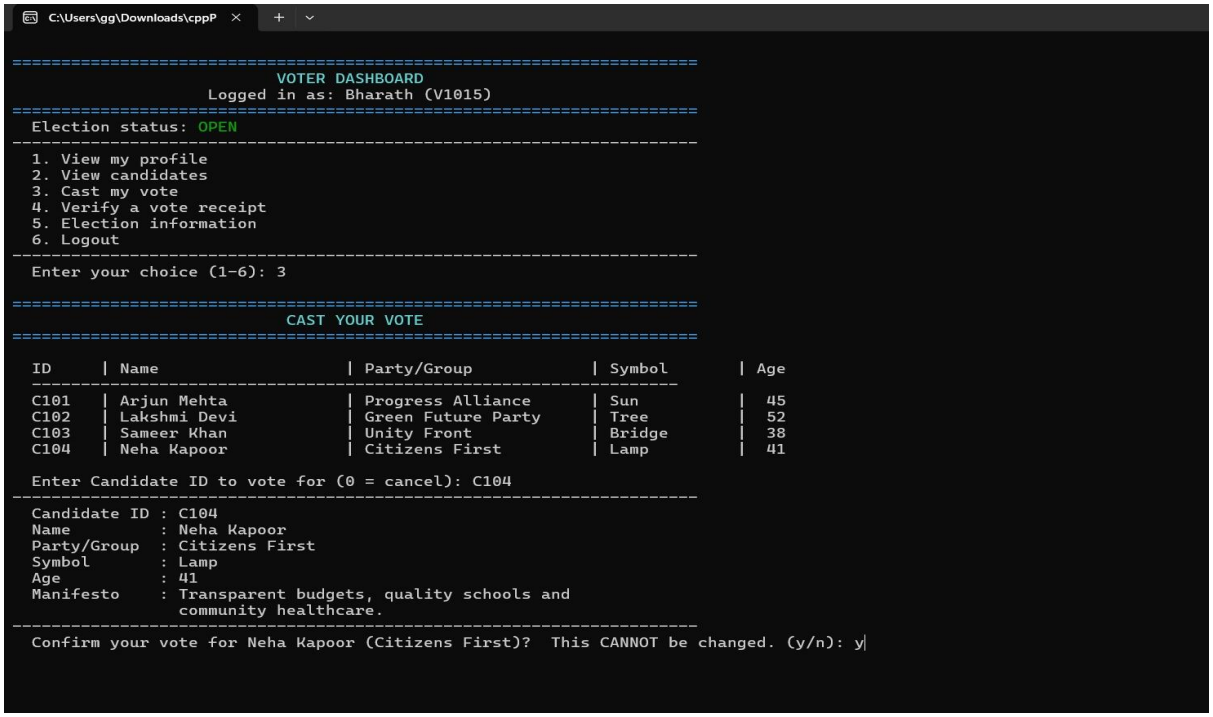
7.6 File Management

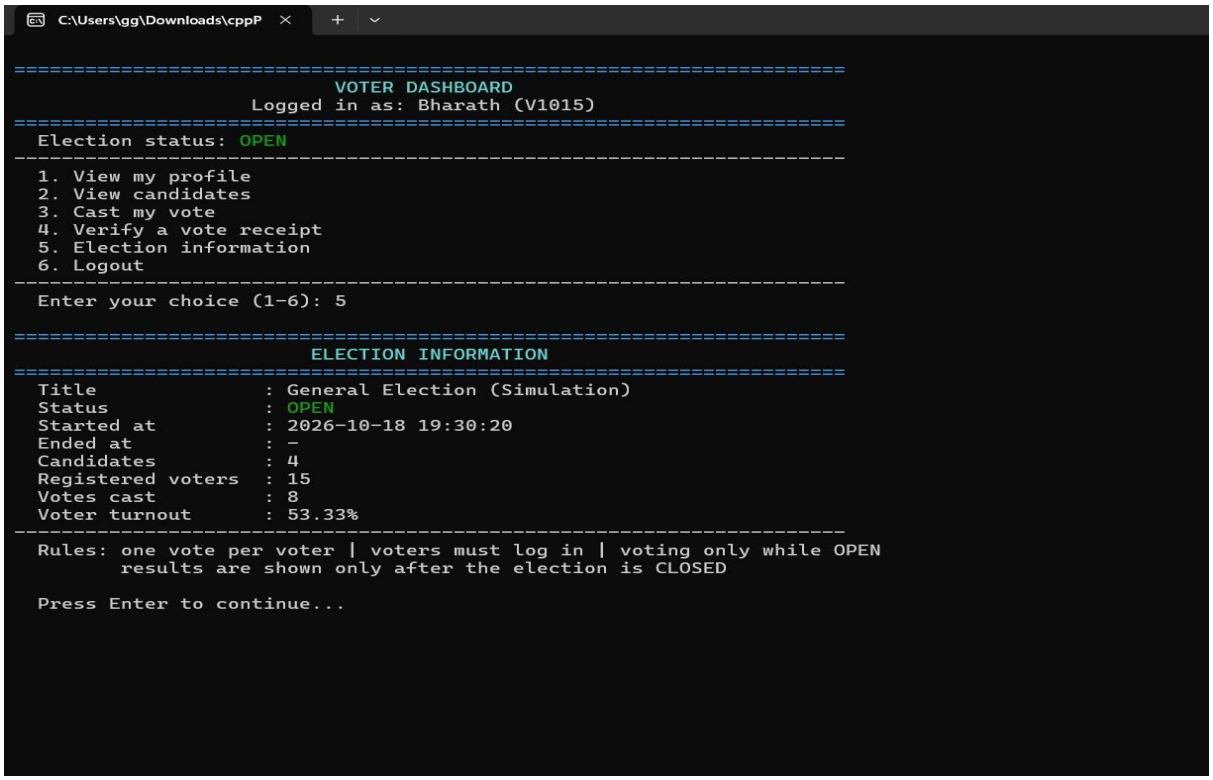
The FileManager handles all reading and writing of text files. Temp files are created and renamed so that if an error occurs the data won’t be partially saved.

Sample Outputs:






C:\Users\gg\Downloads\cppP
VOTER DASHBOARD
Logged in as: Bharath (V1015)
Election status: OPEN
1. View my profile
2. View candidates
3. Cast my vote
4. Verify a vote receipt
5. Election information
6. Logout
Enter your choice (1-6): 3
CAST YOUR VOTE
ID | Name | Party/Group | Symbol | Age
C101 | Arjun Mehta | Progress Alliance | Sun | 45
C102 | Lakshmi Devi | Green Future Party | Tree | 52
C103 | Sameer Khan | Unity Front | Bridge | 38
C104 | Neha Kapoor | Citizens First | Lamp | 41
Enter Candidate ID to vote for (0 = cancel): C104
Candidate ID : C104
Name : Neha Kapoor
Party/Group : Citizens First
Symbol : Lamp
Age : 41
Manifesto : Transparent budgets, quality schools and community healthcare.
Confirm your vote for Neha Kapoor (Citizens First)? This CANNOT be changed. (y/n): y


C:\Users\gg\Downloads\cppP
VOTER DASHBOARD
Logged in as: Bharath (V1015)
Election status: OPEN
1. View my profile
2. View candidates
3. Cast my vote
4. Verify a vote receipt
5. Election information
6. Logout
Enter your choice (1-6): 5
ELECTION INFORMATION
Title : General Election (Simulation)
Status : OPEN
Started at : 2026-10-18 19:30:20
Ended at : -
Candidates : 4
Registered voters : 15
Votes cast : 8
Voter turnout : 53.33%
Rules: one vote per voter | voters must log in | voting only while OPEN
results are shown only after the election is CLOSED
Press Enter to continue...

8. OOP Concepts

8.1 Classes and Objects

The classes in this project represent entities within the application such as Candidate, Voter, Admin and Election. Objects are created from the classes to interact with real data.

8.2 Encapsulation

Important data members are made private or protected and can only be altered using functions. This is demonstrated by the hasVoted member in the voter class. This can't be altered directly and can only be changed using the markVoted() function. This will stop users from voting twice.

8.3 Inheritance

The Voter and Admin classes inherit common properties and behaviours from the User class. This prevents duplication of code.

8.4 Abstraction

The User class is abstract. It contains common interfaces and can't be instantiated directly. This is because the application deals with specific user types such as voters.

8.5 Polymorphism

This application uses pointers to User objects. Virtual functions such as role() are used. Depending on whether it is a Voter or Admin object the correct function will be called.

8.6 Constructors and Destructor

Constructors with member initializer lists are used. The User class has a virtual destructor to allow the derived objects to be destroyed correctly.

8.7 STL Containers & Algorithms

The application uses a map to store voters, vector to store candidates and votes, set to check receipts, and optional for the administrator. Algorithms are used such as sort, copy_if, remove_if, shuffle and transform.

9. Searching & Sorting

9.1 Binary Search

Candidate IDs are kept in order. The findCandidate() function takes an ID such as C102 and extracts the numeric portion. The search is then narrowed down. Since the candidates are ordered by the same key the candidate can be found in O(log n).

9.2 Linear Search

Searching for candidates and voters with text fields uses linear search. This is useful for searching for substrings regardless of case since there may be multiple fields.

9.3 Sorting

The application can sort voters by ID, name or status; candidates by ID, name or party and results by vote count and then name. Lambdas are used to specify how they should be sorted.

10. File Handling

The application stores data using text files. The data is parsed using ifstream and ofstream. Any missing files can be recreated. If the data is invalid, it will be skipped, and a warning will show.

To make saving more secure the application writes to a temp file and then renames it.

11. Exceptions & Validation

Exceptions are thrown in the logic layer using custom exceptions that derive from VotingException. These are caught in the console layer and a friendly message is displayed. This means that the user can continue using the menu. Any unexpected standard exceptions are handled at the top.

Data is read in as full lines. Numeric data is checked to ensure it is in the correct range. Other data is validated using validation functions to prevent common errors.

12. Security & Privacy

This project has tried to create a secret ballot. The vote doesn’t contain any informations about the user. It contains the receipt, candidate and date. The audit log

only shows voting activity and doesn't show what candidate was chosen or the receipt. Once the election has CLOSED the ballots are shuffled so the order in the file won't match the audit log.

The passwords are hashed using the salted FNV-1a hash. This isn't suitable for a real security system. Strong hashing algorithms should be used such as bcrypt or Argon2. The data files are plain text and can be altered by anyone with access to the files.

13. Crash Safety & Data Integrity

The voting process will add the ballot and then mark the user as having voted. If the application crashes between these steps it can be spotted using the data integrity. Duplicate receipts, unknown candidates, too many ballots and ballots while the election hasn't started will be checked for.

Candidates can only be edited and voters deleted when the election is in the NOT STARTED status. This prevents the candidate list and electorate from being changed.

14. Testing

The finished project has been designed to be tested manually and with scripts. Some important cases to test include successful registration/login, invalid login details, duplicate voting, invalid candidates, voting when the election isn't OPEN, results before closing, missing files, corrupt data and invalid menu inputs.

The election status transitions will be checked, receipts will be checked to ensure they are unique, results will be checked to ensure tied cases are handled and the data integrity will be checked.

15. Results & Observations

The finished application performs as expected with an offline election. Users can register and verify, the administrator can manage election data, voters can cast one verified vote and the application can calculate turnout and results once the election has closed.

The application shows how OOP concepts can be combined with real programming techniques such as STL containers, file handling, searching, sorting and exceptions.

16. Advantages

Runs locally.

Separation of concerns between the user, election, data and interface.

Prevents duplicate voting with multiple checks.

Data is stored between runs thanks to files.

Handles invalid inputs and multiple file issues without crashing.

Shows a wide variety of C++17 and OOP concepts.

17. Limitations

An educational tool.

Passwords are protected using the FNV-1a hash, which isn't a production-grade password-hashing algorithm.

Passwords are visible in the console.

Data is stored as plain text.

No internet, multi-machine or hardware security.

Only single choice plurality voting.

18. Enhancements

Add hidden password entry and stronger password hashing such as bcrypt or Argon2.

Encrypt the data files and improve the audit log security.

Add an SQLite database using an abstract Storage.

Allow multiple elections/constituencies and extra voting methods.

Automatically open/close based on dates/times.

Export reports as CSV/PDF.

Add a GUI/web interface but keep the core VotingSystem.

Add unit testing using frameworks such as Catch2 or GoogleTest.

Use smart pointers, templates and re-usable repository.

Add language files and support for multiple polling booths.

Automatically backup the data directory with dates.

21. Detailed Workflow

21.1 Initialisation

When the application is first opened it will check if the data files exist. If they don't it can recreate the information. At the right time the project can restore demo data. This makes the application easier to test since the user doesn't have to manually create all the records.

The system will also determine what the current election status is and load any existing administrator, voters, candidates and votes. It will try to load the data even if some of the files are missing or invalid.

21.2 Register Voter

When a new voter registers they will enter the requested information. The data will be validated and then an account created. The voter will be stored so it can be used in future sessions. The register process is separated from the voting process so the voter list can be managed separately.

A useful feature of this design is that the register process can be used until the election has CLOSED. The existing electorate can only be changed based on the election status.

21.3 Login

The login process will check if the entered user information matches the stored account information. The password isn't stored directly but is based on a salted FNV-1a hash. Once the user has been verified the application knows if it is a voter/administrator and can provide the correct options.

It is important that the login is separated from the rest of the system. The user shouldn't be able to perform any election actions without being verified.

21.4 Voting

The user will login and enter the voting section. The application will check if the election is OPEN and if the user has voted. It will also check if the candidate exists. The user will be asked to confirm their choice.

Once confirmed the vote will be added to the vote storage and the user will be marked as having voted. A unique receipt will be generated. This can be used to confirm that the vote exists but isn't intended to show what candidate was chosen.

21.5 Close Election

Once the voting has finished the administrator can change the election status to CLOSED. Results can now be viewed. The application can calculate the turnout, votes per candidate, highest vote count and tie.

The vote records will be shuffled once the election has stopped. This is an attempt to stop an obvious relationship between the audit information and the vote file.

22. Detailed Class Responsibilities

22.1 User

The User class is an abstract base class for the accounts. It contains common account information and declares virtual functions that the derived classes must provide. It is an abstract class to stop an account being created without a specific role.

This class demonstrates protected data members, virtual functions, virtual destructors and override in the derived classes.

22.2 Voter

This class represents a user that can take part in the election. It contains the parent class account information along with the voting status. The markVoted() function ensures that only one vote can be cast by not allowing the user to vote again once the status has been changed.

The user can view their profile and access candidate information.

19. Conclusion

The Offline Election and Voting Management System has given a practical application of C++17 and OOP concepts. The project uses inheritance, abstraction, encapsulation

and polymorphism along with file handling, STL containers, searching, sorting, exceptions and validation.

The main thing I learned from this project was not just creating the individual classes but how they should all work together. Splitting up the VotingSystem, ConsoleUI and FileManager made in the program easier to manage. It also showed how to apply separation of concerns in a real project. It highlighted how this is an educational tool and not a production system, especially regarding password security, data and verification.

20. References

C++17 Standard Library documentation and language references.

Course material on OOP, data structures and algorithms.

Project code and test cases.